

page.tsx

frontend/app | TypeScript / React

```
/**
 * page.tsx (or pages/index.tsx)
 */
// BioSync main dashboard — the Human OS interface.
// Layout:
// ████████████████████████████████████████████████████████████████
// ■ BIOSYNC_v3 // HUMAN_OS [confidence] ■
// ████████████████████████████████████████████████████████████████
// ■ WEBCAM FEED ■ VPG WAVEFORM ■
// ████████████████████████████████████████████████████████████████
// ■ COGNITIVE SCORE ■ STATE INDICATOR ■
// ████████████████████████████████████████████████████████████████
// ■ AI INTERVENTION BOX ■
// ████████████████████████████████████████████████████████████████
// ■ FAILURE MEMORY LOG (sidebar on critical) ■
// ████████████████████████████████████████████████████████████████
//
// Color system (drives every visual element):
// CALM → cyan-500 (#22d3ee)
// LOAD → yellow-400 (#facc15)
// CRITICAL → red-500 (#ef4444)
*/

'use client';

import { useState, useEffect, useRef, useCallback } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import VpgGraph from '../components/VpgGraph';
import { useFusionEngine } from '../hooks/useFusionEngine';
import { useStateMemory } from '../hooks/useStateMemory';
import { parseHinglishIntent } from '../utils/hinglishParser';
import { getBioSyncIntervention } from '../ai/groqClient';
import type { CognitiveState } from '../utils/stressEngine';

// ████ Theme ████████████████████████████████████████████████████████████████

const STATE_THEME: Record<CognitiveState, {
border: string;
text: string;
bg: string;
shadow: string;
badge: string;
glow: string;
}> = {
CALM: {
border: 'border-cyan-500',
text: 'text-cyan-400',
bg: 'bg-cyan-500/10',
shadow: 'shadow-cyan-500/20',
badge: 'bg-cyan-500/20 text-cyan-300',
glow: 'shadow-cyan-500/30',
},
LOAD: {
border: 'border-yellow-400',
text: 'text-yellow-400',
bg: 'bg-yellow-400/10',
shadow: 'shadow-yellow-400/20',
badge: 'bg-yellow-400/20 text-yellow-300',
glow: 'shadow-yellow-400/30',
},
CRITICAL: {
border: 'border-red-500',
```



```

<div className="space-y-4">
<div className={`border rounded-xl p-4 transition-all duration-700 ${theme.border} ${theme.shadow}
shadow-lg`}>
<p className="text-xs text-slate-500 mb-2">WEBCAM_FEED</p>
<video
ref={videoRef}
autoPlay
muted
playsInline
className="w-full rounded-lg bg-slate-900 aspect-video object-cover"
/>
<canvas ref={canvasRef} width={320} height={240} className="hidden" />
</div>

{/* Channel breakdown */}
<div className="border border-slate-800 rounded-xl p-4 space-y-3">
<p className="text-xs text-slate-500">SIGNAL_CHANNELS</p>
[[
{ label: 'VISUAL (rPPG)', value: channels.visual, weight: '×0.3', color: 'bg-cyan-500' },
{ label: 'AUDIO (jitter)', value: channels.audio, weight: '×0.5', color: 'bg-yellow-400' },
{ label: 'SEMANTIC (NLP)', value: channels.semantic, weight: '×0.2', color: 'bg-purple-500' },
].map(ch => (
<div key={ch.label}>
<div className="flex justify-between text-xs text-slate-400 mb-1">
<span>{ch.label}</span>
<span className="text-slate-600">{ch.weight} – {ch.value}</span>
</div>
<div className="h-1.5 bg-slate-800 rounded-full overflow-hidden">
<motion.div
className={`h-full rounded-full ${ch.color}`}
animate={{ width: `${ch.value}%` }}
transition={{ duration: 0.5 }}
/>
</div>
</div>
)))
</div>
</div>

{/* Column 2: VPG + score */}
<div className="col-span-1 space-y-4">
<div className={`border rounded-xl p-4 transition-all duration-700 ${theme.border}`}>
<p className="text-xs text-slate-500 mb-2">VASCULAR_PRESSURE_GRAPH (rPPG)</p>
<VpgGraph
stressLevel={assessment.score}
isOffline={isOffline}
state={assessment.state}
signalConfidence={signalConfidence}
/>
</div>

{/* Cognitive score */}
<div className={`border rounded-xl p-6 transition-all duration-700 ${theme.border}
${assessment.state === 'CRITICA...`}}>
<p className="text-xs text-slate-500 mb-3">COGNITIVE_SCORECARD</p>
<motion.div
className={`text-7xl font-black tabular-nums ${theme.text}`}
key={Math.floor(assessment.score)}
animate={{ scale: [1, 1.05, 1] }}
transition={{ duration: 0.3 }}
>
{Math.round(assessment.score)}
</motion.div>
<div className={`text-xl font-bold mt-2 ${theme.text}`}>
{assessment.state}_STATE
</div>
<div className={`text-xs mt-1 px-2 py-0.5 rounded inline-block ${theme.badge}`}>
{assessment.state === 'CALM' && 'PLANNING_MODE'}

```


[illegible]